

Fleet Observability & Incident Investigation

Executive Summary Report | Author: Ruthwik Thotapelli | Role: Data Analytics & Observability Intern | July 2026
Fleet: Autonomous Mobile Robot (AMR) Operations | Incident Window: 10:00 AM – 11:00 AM, June 30, 2026

1. Project Objective

To conduct a rigorous, data-driven postmortem on a critical operational incident affecting a fleet of Autonomous Mobile Robots (AMRs). The goal was to move beyond assumptions and use **raw telemetry, trip lifecycle records, and system event logs** to precisely reconstruct the failure sequence, identify all anomalies across the fleet, and conclusively prove the root cause of the mission-critical trip cancellation.

2. Approach

Phase	Script	Description
1. Data Profiling	preprocessing.py	Assessed dataset health — checked missing values, timestamp integrity, and row uniqueness across all three data sources before any analysis.
2. Exploratory Analysis	exploratory_analysis.py	Visualized CPU, battery, and network latency distributions fleet-wide to isolate statistical outliers.
3. Timeline Reconstruction	incident_timeline.py	Fused three asynchronous data streams (telemetry, trips, events) into a single, second-level chronological timeline.
4. Root Cause Extraction	root_cause_analysis.py	Applied rule-based pattern matching on the fused timeline to link prior events to trip cancellation outcomes.

3. Incident Timeline (10:00 AM – 11:00 AM)

Time	Robot	Event	Details
10:05:12	Robot_01	Trip T101 Started	Source → Destination
10:05:14	Robot_01	Resource Lock Acquired	Zone_A secured ✓
10:05:55	Robot_01	■ Battery Low Warning	Pre-trip check missed
10:06:10	Robot_01	■ Network Latency Spike	1450 ms sustained
10:06:18	Robot_01	■ Resource Lock Timeout	Zone_B — failed to acquire
10:06:20	Robot_01	■ Trip T101 CANCELLED	Safety timeout triggered
10:15:00	Robot_02	Trip T102 Started	Duplicate log entry observed
10:17:40	Robot_03	Trip T204 Started	Zone_C lock acquired
10:18:05	Robot_03	■ Position Jump Detected	Coords: X:999, Y:999
10:18:15	Robot_03	Trip T204 Completed	Self-recovered ✓
10:21:30	Robot_02	■ CPU Saturation	95% CPU load
10:24:00	Robot_02	Trip T102 Completed	Resilient under compute load ✓
10:40:00	Robot_04	Trip T301 Started	Normal start
10:44:00	Robot_04	■ Battery Level Jump	32% → 41% (impossible mid-trip)

■ **Critical Cascade:** The 1450ms network spike at 10:06:10 directly caused the Zone_B lock timeout 8 seconds later, which triggered the automatic safety mechanism cancelling Trip T101. The **entire failure chain spanned only 10 seconds**.

4. Five Detected Anomalies

ANOMALY 1 — Network Latency Spike (Robot_01)

Observed: Network latency of **1450 ms** at 10:06:10, confirmed in both event logs and telemetry.

Threshold: Normal ops require <200 ms for lock acquisition RPC calls.

Impact: **Mission-critical.** Directly caused Trip T101 cancellation.

ANOMALY 2 — CPU Saturation (Robot_02)

Observed: CPU utilization reached **95%** at 10:21:30, well above the 90% alert threshold.

Impact: Non-critical. Robot_02 successfully completed Trip T102, demonstrating edge-software resilience.

ANOMALY 3 — SLAM Localization Glitch (Robot_03)

Observed: Telemetry logged coordinates **X:999, Y:999** at 10:18:05 — a sentinel value indicating an unhandled null or out-of-bounds SLAM output.

Impact: Non-critical. Robot self-recovered within 10 seconds and completed Trip T204.

ANOMALY 4 — Impossible Battery Jump (Robot_04)

Observed: Battery level rose from **32% → 41%** during an active running trip at 10:44:00. Confirmed by a BatteryLevelJump event log entry.

Impact: Non-critical operationally, but **hardware-critical**. AMRs cannot charge while in motion. This isolates a defective Battery Management System (BMS) sensor requiring replacement.

ANOMALY 5 — Event Log Idempotency Bug (Robot_02)

Observed: The TripStarted event for Trip T102 was logged **twice** at exactly 10:15:00.

Impact: Non-critical operationally, but indicates a bug in the telemetry ingestion pipeline that could corrupt downstream analytics, aggregations, and billing counts.

5. Root Cause

Root Cause: A severe network latency spike (1450ms) on Robot_01 prevented it from communicating with the resource management server within the required timeout window. The server, unable to receive lock acquisition confirmation for Zone_B, triggered an automatic safety timeout and cancelled Trip T101. Compute was stable (54% CPU). Battery was NOT the cause. The failure was purely a network-layer issue.

Confirmed Evidence Chain:

1. NetworkLatencyHigh (1450ms) logged at **10:06:10**
2. ResourceLockTimeout (Zone_B) logged **8 seconds later** at 10:06:18
3. TripCancelled (T101) logged **2 seconds later** at 10:06:20

6. Additional Observation — Hidden Safety Risk

■ Critical Safety Design Flaw Identified

At 10:05:55, Robot_01 generated a **BatteryLowWarning** — and yet the system allowed the robot to continue its trip, acquire Zone_A, and attempt to acquire Zone_B.

Why this is dangerous: In a warehouse environment, if a low-battery robot acquires a lock on a shared critical zone (e.g., a central intersection) and its battery dies mid-transit, it creates a **physical blockade**. Other robots cannot enter that zone until the lock is manually released and the robot is physically removed — potentially shutting down the **entire fleet's throughput**.

Recommendation: Implement a *pre-trip battery gate* — any robot below a configurable threshold (e.g., 30%) should be barred from acquiring new zone locks and immediately routed to a charging dock.

7. Conclusion

This investigation conclusively identified a **network-induced resource lock timeout** as the sole cause of the mission-critical Trip T101 cancellation. The four secondary anomalies (CPU saturation, SLAM glitch, BMS sensor fault, and log duplication) were isolated to individual robots and did not affect fleet-wide operations — a testament to the robustness of the edge-deployed robot software.

Finding	Status	Action Required
Robot_01 Trip Cancellation	Root cause proven	Network SLA audit + redundancy review
Robot_02 CPU Saturation	Edge software resilient	Monitor; set capacity alerts
Robot_03 SLAM Glitch	Self-recovered	Improve SLAM null handling
Robot_04 BMS Sensor Fault	Hardware defect	BMS replacement & calibration
Pipeline Idempotency Bug	Software defect	Fix ingestion deduplication logic
Low-battery lock policy	SAFETY RISK	Implement pre-trip battery gate — URGENT

Key Takeaway: The most urgent systemic risk is not the network incident itself, but the **absence of a battery-level pre-check** before zone lock acquisition — a policy gap that could cause a significantly more severe operational disruption if left unaddressed.